Big-O Notation

Prof. Timothy Sun

Big-O Notation

$$
\begin{aligned}
&\textsc{InsertionSort}(A[1,\ldots,n]):\\
&\quad \textbf{for } i = 2,\ldots,n\\
&\qquad\quad key := A[i]\\
&\qquad\quad j := i-1\\
&\qquad\quad \textbf{while } j > 0 \text{ and } A[j] > key\\
&\qquad\qquad\quad A[j+1] := A[j]\\
&\qquad\qquad\quad j := j-1\\
&\qquad\quad A[j+1] := key
\end{aligned}
$$

"Insertion sort runs in O($\boldsymbol{n}^2$) time."

Why use Big-O notation?

- Language/machine/unit-independent
- Isolating the “bottleneck”
- Captures long-term behavior
- Easier to analyze

Why use Big-O notation?

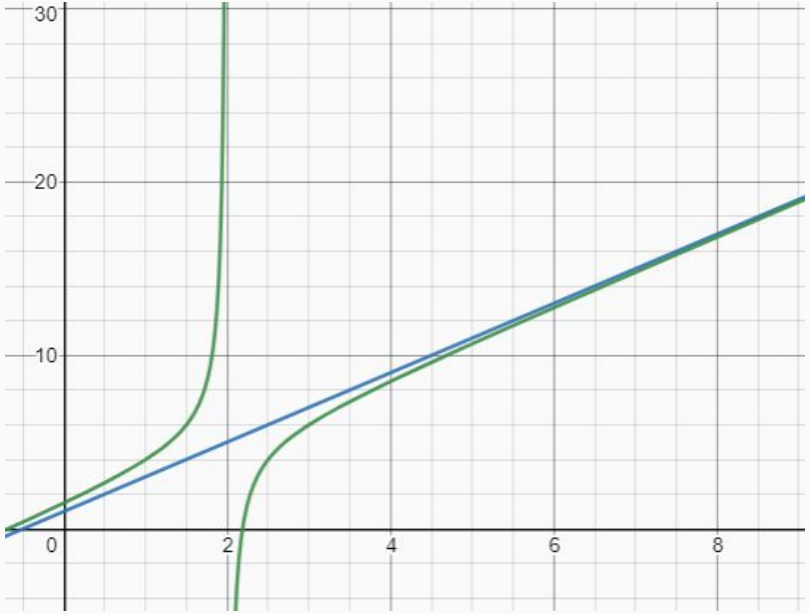
- Language/machine/unit-independent
- Isolating the “bottleneck”
- Captures long-term behavior
- Easier to analyze

Big-O notation is about ***scalability***, not speed!

Asymptotes

$$f(x) = 2x + 1$$

$$g(x) = \frac{2x^2 - 3x - 3}{x - 2}$$



Big-O notation

O(f(***n***)): ***class*** of functions whose “order is ***at most*** f(***n***).”

Big-O notation

O(f(***n***)): ***class*** of functions whose “order is ***at most*** f(***n***).”

$$f(n) \in O(g(n))$$

Big-O notation

O(f(***n***)): ***class*** of functions whose “order is ***at most*** f(***n***).”

$$f(n) \in O(g(n))$$

More often you’ll see it (technically, incorrectly) written as:

$$\text{“}f(n) = O(g(n))\text{”}$$

Big-O notation

Definition: $f(\boldsymbol{x}) = O(g(\boldsymbol{x}))$, i.e., "$f(\boldsymbol{x})$ is order *at most* $g(\boldsymbol{x})$", if:

There ***exist*** numbers $\boldsymbol{C}, \boldsymbol{K} > 0$ so that

$$f(\boldsymbol{x}) \leq \boldsymbol{C} \cdot g(\boldsymbol{x}),$$

for every $\boldsymbol{x} \geq \boldsymbol{K}$.

Big-O notation

Definition: $f(\boldsymbol{x}) = O(g(\boldsymbol{x}))$, i.e., "$f(\boldsymbol{x})$ is order *at most* $g(\boldsymbol{x})$", if:

There ***exist*** numbers $\boldsymbol{C}, \boldsymbol{K} > 0$ so that

$$f(\boldsymbol{x}) \leq \boldsymbol{C} \cdot g(\boldsymbol{x}),$$

for every $\boldsymbol{x} \geq \boldsymbol{K}$.

C: use it to hide the coefficient
K: only care about "big enough" ***x***

Big-O notation

Some examples:

- $3\boldsymbol{x} + 4 = O(\boldsymbol{x})$
- $4\boldsymbol{x}^2 + 10\boldsymbol{x} - 21 = O(\boldsymbol{x}^2)$
- $81\boldsymbol{x} = O(\boldsymbol{x}^3)$
- $\boldsymbol{x}^2 \neq O(\boldsymbol{x})$
- $\log_2 \boldsymbol{x} = O(\ln \boldsymbol{x})$ [base doesn't matter]
- $\boldsymbol{x}^{100} = O(2^{\boldsymbol{x}})$
- $2^{\boldsymbol{n}} = O(\boldsymbol{n}!)$

A linear example

One approach is to “guess” $\boldsymbol{C}$, then find $\boldsymbol{K}$ algebraically: e.g. $3\boldsymbol{x} + 4 = O(\boldsymbol{x})$

A linear example

One approach is to “guess” ***C***, then find ***K*** algebraically: e.g. $3\boldsymbol{x} + 4 = O(\boldsymbol{x})$

1. “Guess” a ***C*** bigger than 3: try ***C*** = 4

A linear example

One approach is to “guess” $\boldsymbol{C}$, then find $\boldsymbol{K}$ algebraically: e.g. $3\boldsymbol{x} + 4 = O(\boldsymbol{x})$

1. “Guess” a $\boldsymbol{C}$ bigger than 3: try $\boldsymbol{C} = 4$
2. Solve for $\boldsymbol{K}$:

$$3\boldsymbol{n} + 4 \leq 4\boldsymbol{x} \rightarrow \boldsymbol{x} \geq 4, \text{ so } \boldsymbol{K} \text{ can be } 4.$$

A linear example

One approach is to “guess” $\boldsymbol{C}$, then find $\boldsymbol{K}$ algebraically: e.g. $3\boldsymbol{x} + 4 = O(\boldsymbol{x})$

1. “Guess” a $\boldsymbol{C}$ bigger than 3: try $\boldsymbol{C} = 4$
2. Solve for $\boldsymbol{K}$:

$$3\boldsymbol{n} + 4 \leq 4\boldsymbol{x} \rightarrow \boldsymbol{x} \geq 4, \text{ so } \boldsymbol{K} \text{ can be } 4.$$

In general, many pairs of *C* and *K* can work.

A quadratic example

$4\boldsymbol{x}^2 + 10\boldsymbol{x} - 21 = O(\boldsymbol{x}^2)$

A quadratic example

$4\boldsymbol{x}^2 + 10\boldsymbol{x} - 21 = O(\boldsymbol{x}^2)$

1. Guess $\boldsymbol{C} = 5$.

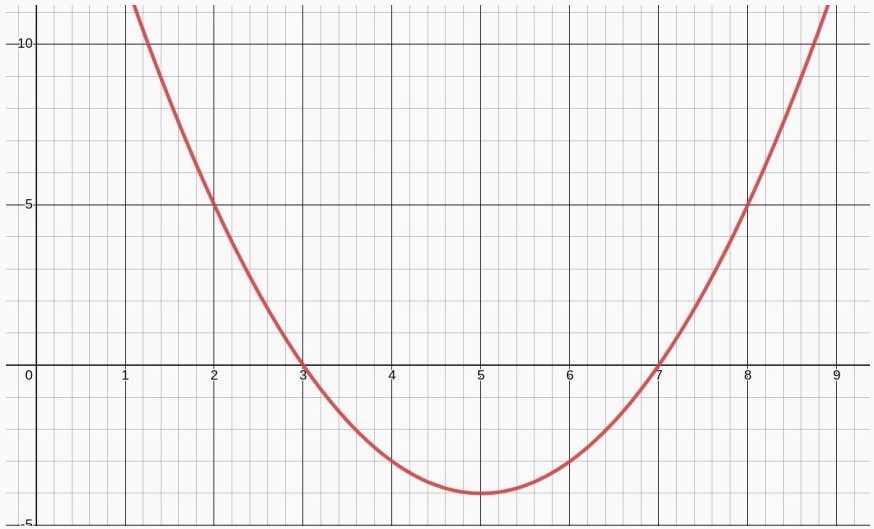
A quadratic example

$4\boldsymbol{x}^2 + 10\boldsymbol{x} - 21 = O(\boldsymbol{x}^2)$

1. Guess $\boldsymbol{C} = 5$.
2. Solve for $\boldsymbol{K}$:

$$4\boldsymbol{x}^2 + 10\boldsymbol{x} - 21 \leq 5\boldsymbol{x}^2 \rightarrow$$

$$0 \leq \boldsymbol{x}^2 - 10\boldsymbol{x} + 21 = (\boldsymbol{x} - 3)(\boldsymbol{x} - 7)$$



A quadratic example

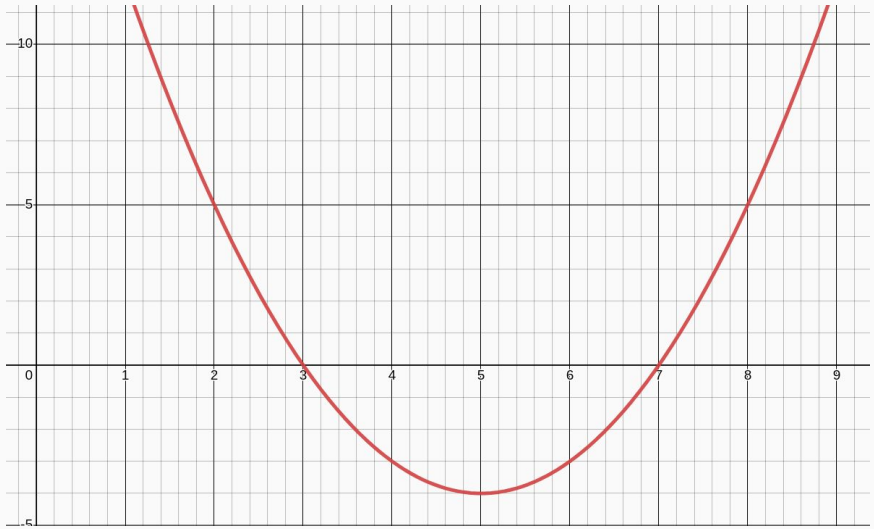
$4\boldsymbol{x}^2 + 10\boldsymbol{x} - 21 = O(\boldsymbol{x}^2)$

1. Guess $\boldsymbol{C} = 5$.
2. Solve for $\boldsymbol{K}$:

$$4\boldsymbol{x}^2 + 10\boldsymbol{x} - 21 \leq 5\boldsymbol{x}^2 \rightarrow$$

$$0 \leq \boldsymbol{x}^2 - 10\boldsymbol{x} + 21 = (\boldsymbol{x} - 3)(\boldsymbol{x} - 7)$$

$\boldsymbol{K} = 7$ works, but not $\boldsymbol{K} = 3$ (why?)



Logarithms

Logarithms under different bases are related by the ***change of base*** formula:

$$\log_a \boldsymbol{x} = \log_b \boldsymbol{x} / \log_b \boldsymbol{x} = (1 / \log_b \boldsymbol{a}) \log_b \boldsymbol{x}$$

Logarithms

Logarithms under different bases are related by the ***change of base*** formula:

$$\log_a x = \log_b x / \log_b x = (1 / \log_b a) \log_b x$$

Inside of big-O notation, ignore the base:

$\underline{O(\log x)}$, not $O(\log_2 x)$

Logarithms

Logarithms under different bases are related by the ***change of base*** formula:

$$\log_a x = \log_b x / \log_b x = (1 / \log_b a) \log_b x$$

Inside of big-O notation, ignore the base:

$\underline{O(\log x)}$, not $O(\log_2 x)$

(unless it appears as an exponent)

General themes

For polynomials, look at the highest power, drop the coefficient.

$$3x^5 + 6x^4 - 10x^3 + x^2 - 15x + 11$$

General themes

For polynomials, look at the highest power, drop the coefficient.

$$3\boldsymbol{x}^5 + 6\boldsymbol{x}^4 - 10\boldsymbol{x}^3 + \boldsymbol{x}^2 - 15\boldsymbol{x} + 11 = O(\boldsymbol{x}^5)$$

Simple approach for polynomials

$$3x^5 + 6x^4 - 10x^3 + x^2 - 15x + 11$$

Simple approach for polynomials

1. Remove negative terms

true for $\boldsymbol{x} \geq 0$

$$3\boldsymbol{x}^5 + 6\boldsymbol{x}^4 - 10\boldsymbol{x}^3 + \boldsymbol{x}^2 - 15\boldsymbol{x} + 11$$

$$\leq \quad 3\boldsymbol{x}^5 + 6\boldsymbol{x}^4 + \boldsymbol{x}^2 + 11$$

Simple approach for polynomials

1. Remove negative terms
2. Raise to highest power

true for $x \geq 0$

true for $x \geq 1$

$$
\begin{aligned}
& 3x^5 + 6x^4 - 10x^3 + x^2 - 15x + 11 \\
\leq\ & 3x^5 + 6x^4 + x^2 + 11 \\
\leq\ & 3x^5 + 6x^5 + x^5 + 11x^5
\end{aligned}
$$

Simple approach for polynomials

1. Remove negative terms
2. Raise to highest power

true for $x \geq 0$

true for $x \geq 1$

$$
\begin{aligned}
& 3x^5 + 6x^4 - 10x^3 + x^2 - 15x + 11 \\
\leq\ & 3x^5 + 6x^4 + x^2 + 11 \\
\leq\ & 3x^5 + 6x^5 + x^5 + 11x^5 \\
=\ & 21x^5
\end{aligned}
$$

Simple approach for polynomials

1. Remove negative terms
2. Raise to highest power

true for $x \geq 0$

true for $x \geq 1$

$K = 1$

$$
\begin{aligned}
& 3x^5 + 6x^4 - 10x^3 + x^2 - 15x + 11 \\
\leq\ & 3x^5 + 6x^4 + x^2 + 11 \\
\leq\ & 3x^5 + 6x^5 + x^5 + 11x^5 \\
=\ & 21x^5
\end{aligned}
$$

c

Polynomials vs. Exponentials

One can interpret big-O notation as a ***limit***

$f(\boldsymbol{x})/g(\boldsymbol{x}) \leq \boldsymbol{C}$, for every $\boldsymbol{x} \geq \boldsymbol{K}$.

Polynomials vs. Exponentials

One can interpret big-O notation as a ***limit***

$f(\boldsymbol{x})/g(\boldsymbol{x}) \leq \boldsymbol{C}$, for every $\boldsymbol{x} \geq \boldsymbol{K}$.

Can use calculus to help evaluate things like

$\boldsymbol{x}^{100} = O(e^{\boldsymbol{x}})$.

Polynomials vs. Exponentials

One can interpret big-O notation as a ***limit***

$f(\boldsymbol{x})/g(\boldsymbol{x}) \leq \boldsymbol{C}$, for every $\boldsymbol{x} \geq \boldsymbol{K}$.

Can use calculus to help evaluate things like

$\boldsymbol{x}^{100} = O(e^{\boldsymbol{x}})$.

$$
\begin{aligned}
\lim_{x\to\infty} \frac{x^{100}}{e^x} &= \lim_{x\to\infty} \frac{100 \cdot x^{99}}{e^x} \\
&= \lim_{x\to\infty} \frac{100 \cdot x^{99}}{e^x} \\
&= \lim_{x\to\infty} \frac{100 \cdot 99 \cdot x^{98}}{e^x} \\
&\vdots \\
&= \lim_{x\to\infty} \frac{\text{constant}}{e^x} \\
&= 0
\end{aligned}
$$

General themes

For polynomials, look at the highest power, drop the coefficient.

$$203n^5 + 92n^4 - 1423n^3 + n^2 + 86n - 571 = O(n^5)$$

General themes

For polynomials, look at the highest power, drop the coefficient.

$$203n^5 + 92n^4 - 1423n^3 + n^2 + 86n - 571 = O(n^5)$$

constant < logarithmic < polynomial < exponential

General themes

For polynomials, look at the highest power, drop the coefficient.

$$203\boldsymbol{n}^5 + 92\boldsymbol{n}^4 - 1423\boldsymbol{n}^3 + \boldsymbol{n}^2 + 86\boldsymbol{n} - 571 = O(\boldsymbol{n}^5)$$

constant < logarithmic < polynomial < exponential

$$\log_{\boldsymbol{b}} \boldsymbol{n} = O(\boldsymbol{n}^{\boldsymbol{d}}), \text{ for any } \boldsymbol{b} > 1, \boldsymbol{d} > 0$$

General themes

For polynomials, look at the highest power, drop the coefficient.

$$203n^5 + 92n^4 - 1423n^3 + n^2 + 86n - 571 = O(n^5)$$

constant < logarithmic < polynomial < exponential

$\log_b n = O(n^d)$, for any $b > 1$, $d > 0$

$n^d = O(c^n)$ for any $d > 0$, $c > 1$.

Useful properties

$n^a = O(n^b)$, if $a \leq b$

Useful properties

$n^a = O(n^b)$, if $a \leq b$

If $f(n) = O(g(n))$, and $h(n) > 0$, then $f(n)h(n) = O(g(n)h(n))$

Useful properties

$n^a = O(n^b)$, if $a \leq b$

If $f(n) = O(g(n))$, and $h(n) > 0$, then $f(n)h(n) = O(g(n)h(n))$

$n = O(n \log n)$, $n \log n = O(n^2)$, $O(n^2) = O(n^3)$, etc.

Useful properties

“$O(f(\boldsymbol{n}))+O(g(\boldsymbol{n})) = O(f(\boldsymbol{n})+g(\boldsymbol{n}))$”

Useful properties

"$O(f(\boldsymbol{n}))+O(g(\boldsymbol{n})) = O(f(\boldsymbol{n})+g(\boldsymbol{n}))$"

- If $a(\boldsymbol{n}) = O(f(\boldsymbol{n}))$ and $b(\boldsymbol{n}) = O(g(\boldsymbol{n}))$, then $a(\boldsymbol{n}) + b(\boldsymbol{n}) = O(f(\boldsymbol{n}) + g(\boldsymbol{n}))$

Useful properties

"$O(f(\boldsymbol{n}))O(g(\boldsymbol{n})) = O(f(\boldsymbol{n})g(\boldsymbol{n}))$"

- If $a(\boldsymbol{n}) = O(f(\boldsymbol{n}))$ and $b(\boldsymbol{n}) = O(g(\boldsymbol{n}))$, then $a(\boldsymbol{n})b(\boldsymbol{n}) = O(f(\boldsymbol{n})g(\boldsymbol{n}))$

Useful when considering the complexity of two distinct steps in an algorithm.

Useful properties

- $O(f(\boldsymbol{n})) + O(g(\boldsymbol{n})) = O(f(\boldsymbol{n}) + g(\boldsymbol{n}))$
- $O(f(\boldsymbol{n}))O(g(\boldsymbol{n})) = O(f(\boldsymbol{n})g(\boldsymbol{n}))$
- $O(\boldsymbol{a} \cdot f(\boldsymbol{n})) = O(f(\boldsymbol{n}))$, for constant $\boldsymbol{a} > 0$.
- $f(\boldsymbol{n}) = O(g(\boldsymbol{n}))$, $g(\boldsymbol{n}) = O(h(\boldsymbol{n})) \rightarrow f(\boldsymbol{n}) = O(h(\boldsymbol{n}))$.
- $O(f(\boldsymbol{n})+g(\boldsymbol{n})) = O(\max(f(\boldsymbol{n}), g(\boldsymbol{n})))$

Simplifying expressions

Give a good big-O estimate of f(***n***) using a simple function

$$f(n) = 20\,(n^2 + n^2 \log_4 n)(4n + 3) + (17 \log_3 n + 19)(n^2 + 2)(\ln n - 3)$$

Simplifying expressions

Give a good big-O estimate of f($\boldsymbol{n}$) using a simple function

$$f(\boldsymbol{n}) = 20\,(\boldsymbol{n}^2 + \boldsymbol{n}^2 \log_4 \boldsymbol{n})(4\boldsymbol{n} + 3) + (17 \log_3 \boldsymbol{n} + 19)(\boldsymbol{n}^2 + 2)(\ln \boldsymbol{n} - 3)$$

$$= O(\boldsymbol{n}^2 \log \boldsymbol{n})O(\boldsymbol{n}) + O(\log \boldsymbol{n})O(\boldsymbol{n}^2)O(\log \boldsymbol{n})$$

$$= O(\boldsymbol{n}^3 \log \boldsymbol{n}) + O(\boldsymbol{n}^2(\log \boldsymbol{n})^2)$$

$$= O(\boldsymbol{n}^3 \log \boldsymbol{n})$$

It’s all Greek to me

$f(\boldsymbol{x}) = \Omega(g(\boldsymbol{x}))$

if there are numbers $\boldsymbol{C}, \boldsymbol{K} > 0$ so that

$$f(\boldsymbol{x}) \geq \boldsymbol{C} \cdot g(\boldsymbol{x}), \text{ whenever } \boldsymbol{x} \geq \boldsymbol{K}.$$

It’s all Greek to me

$f(\boldsymbol{x}) = \Omega(g(\boldsymbol{x}))$

if there are numbers $\boldsymbol{C}, \boldsymbol{K} > 0$ so that

$f(\boldsymbol{x}) \geq \boldsymbol{C} \cdot g(\boldsymbol{x})$, whenever $\boldsymbol{x} \geq \boldsymbol{K}$.

$\boldsymbol{x}^3 - 3\boldsymbol{x} + 10 = \Omega(\boldsymbol{x}^2)$

It’s all Greek to me

$f(\boldsymbol{x}) = \Omega(g(\boldsymbol{x}))$

if there are numbers $\boldsymbol{C}, \boldsymbol{K} > 0$ so that

$f(\boldsymbol{x}) \geq \boldsymbol{C} \cdot g(\boldsymbol{x})$, whenever $\boldsymbol{x} \geq \boldsymbol{K}$.

$\boldsymbol{x}^3 - 3\boldsymbol{x} + 10 = \Omega(\boldsymbol{x}^2)$

(could also think of as $g = O(f)$)

It’s all Greek to me

$f(\boldsymbol{x}) = \Theta(g(\boldsymbol{x}))$

if there are numbers $\boldsymbol{C}_1, \boldsymbol{C}_2, \boldsymbol{K} > 0$ so that

$$\boldsymbol{C}_1 g(\boldsymbol{x}) \leq f(\boldsymbol{x}) \leq \boldsymbol{C}_2 g(\boldsymbol{x}), \text{ whenever } \boldsymbol{x} \geq \boldsymbol{K}.$$

It’s all Greek to me

$f(\boldsymbol{x}) = \Theta(g(\boldsymbol{x}))$

if there are numbers $\boldsymbol{C}_1, \boldsymbol{C}_2, \boldsymbol{K} > 0$ so that

$$\boldsymbol{C}_1 g(\boldsymbol{x}) \leq f(\boldsymbol{x}) \leq \boldsymbol{C}_2 g(\boldsymbol{x}), \text{ whenever } \boldsymbol{x} \geq \boldsymbol{K}.$$

(can also think of as $f = O(g)$ and $f = \Omega(g)$)

It’s all Greek to me

Let $f(\boldsymbol{x}) = \boldsymbol{x} \ln \boldsymbol{x}$, $g(\boldsymbol{x}) = \boldsymbol{x}(\boldsymbol{x} + 1)/2$, $\boldsymbol{h}(x) = (\boldsymbol{x} - 1)^3$

It’s all Greek to me

Let $f(\boldsymbol{x}) = \boldsymbol{x} \ln \boldsymbol{x}$, $g(\boldsymbol{x}) = \boldsymbol{x}(\boldsymbol{x} + 1)/2$, $\boldsymbol{h}(x) = (\boldsymbol{x} - 1)^3$

- $f(\boldsymbol{x}) = O(\boldsymbol{x}^2)$, $g(\boldsymbol{x}) = O(\boldsymbol{x}^2)$, $h(\boldsymbol{x}) \neq O(\boldsymbol{x}^2)$
- $f(\boldsymbol{x}) \neq \Omega(\boldsymbol{x}^2)$, $g(\boldsymbol{x}) = \Omega(\boldsymbol{x}^2)$, $h(\boldsymbol{x}) = \Omega(\boldsymbol{x}^2)$

It’s all Greek to me

Let $f(\boldsymbol{x}) = \boldsymbol{x} \ln \boldsymbol{x}$, $g(\boldsymbol{x}) = \boldsymbol{x}(\boldsymbol{x} + 1)/2$, $\boldsymbol{h}(x) = (\boldsymbol{x} - 1)^3$

- $f(\boldsymbol{x}) = O(\boldsymbol{x}^2)$, $g(\boldsymbol{x}) = O(\boldsymbol{x}^2)$, $h(\boldsymbol{x}) \neq O(\boldsymbol{x}^2)$
- $f(\boldsymbol{x}) \neq \Omega(\boldsymbol{x}^2)$, $g(\boldsymbol{x}) = \Omega(\boldsymbol{x}^2)$, $h(\boldsymbol{x}) = \Omega(\boldsymbol{x}^2)$

Putting these together, we get:

- $f(\boldsymbol{x}) \neq \Theta(\boldsymbol{x}^2)$, $g(\boldsymbol{x}) = \Theta(\boldsymbol{x}^2)$, $h(\boldsymbol{x}) \neq \Theta(\boldsymbol{x}^2)$

Only $g(\boldsymbol{x})$ has quadratic growth.

It’s all Greek to me

O: “≤”

Ω: “≥”

Θ: “=”

It’s all Greek to me

Ο: “≤”

Ω: “≥”

Θ: “=”

ο: “<”

ω: “>”

Concluding Remarks

Big-O notation is useful for measuring the **scalability** of **algorithms**, independent of any implementation…

Concluding Remarks

Big-O notation is useful for measuring the **scalability** of **algorithms**, independent of any implementation…

...but it doesn't always tell the whole story because of the **hidden constant**.

- e.g. mergesort ($O(\boldsymbol{n} \log \boldsymbol{n})$) vs. insertion sort ($O(\boldsymbol{n}^2)$) for small values.

Concluding Remarks

Big-O notation is useful for measuring the **scalability** of **algorithms**, independent of any implementation…

...but it doesn't always tell the whole story because of the **hidden constant**.

- e.g. mergesort ($O(\boldsymbol{n} \log \boldsymbol{n})$) vs. insertion sort ($O(\boldsymbol{n}^2)$) for small values.

Not a substitute for benchmarking!

Concluding Remarks

"Unfortunately, for any instance [graph] that one could fit into the known universe, one would easily prefer $\boldsymbol{n}^{70}$ to even ***constant*** time, if that constant had to be one of Robertson and Seymour's."

-David Johnson, *The NP-Completeness Column: An Ongoing Guide* (1987).